

CdataControl.c4.70 KIB

```
1  #include <gtk/gtk.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include "commonType.h"
5  #include "dataControl.h"
6  #include "initAndTerm.h"
7  #include <pthread.h>
8
9  extern int getData(void);
10 pthread_t pThread;
11 THREAD_STATE existThr = THREAD_DISABLE;
12
13 guint tid;
14 int num = 0;
15
16 void selectPrint(Node** root){
17     printf("\n|선택|\n");
18     printNode(root);
19 }
20
21 void selectInsert(Node** root){
22     int num = 0;
23     Node* insNode = NULL;
24     Node* checkNode = NULL;
25
26     num = getData();
27     checkNode = searchSameNode(root, num);
28     if(checkNode != NULL){
29         printf("\n중복되는 번호입니다.\n");
30         return;
31     }
32
33     insNode = createNode(num);
34     insertNode(root, insNode);
35 }
36
37 void selectDelete(Node** root){
38     int num = 0;
39     Node* checkNode = NULL;
40     num = getData();
41     checkNode = searchSameNode(root, num);
42     if(checkNode == NULL){
43         printf("\n없는 번호입니다.\n");
44         return;
45     }
46     deleteNode(root, num);
47 }
48
49 void selectEnd(Node** root){
50     releaseNode(root);
51 }
52
53 void selectInterConnect() {
54     createThread();
55 }
56
57 void selectTransfer() {
58     transferData();
59 }
60
61 void createThread() {
62     int param = 0;
63     int thrCreateRes = 0;
64
65     if (existThr == THREAD_ENABLE) {
66         gtk_main_quit();
67         usleep(500000);
68         pthread_cancel(pThread);
69         existThr = THREAD_DISABLE;
70     }
71     else {
72         thrCreateRes = pthread_create(&pThread, NULL
73                                     , threadFunc, (void*)&param);
74         pthread_detach(pThread);
75         existThr = THREAD_ENABLE;
76     }
77 }
78
79 void* threadFunc(void* threadFuncArg) {
80     int cleanUpParam = 0;
81
82     gtk_main();
83     pthread_cleanup_push(threadCleanUp, (void *)&cleanUpParam);
84     while (1) {
85         if (existThr == THREAD_DISABLE) {
86             break;
87         }
88     }
89     pthread_exit(NULL);
90     pthread_cleanup_pop(0);
91 }
92
93
94 void threadCleanUp(void *cleanUpArg) {
95     printf("\n %s %d\n",__FUNCTION__, *((int*)cleanUpArg));
96 }
97
98 void printNode(Node** root){
99     Node* temp = *root;
100     if(temp == NULL){
101         printf("\n");
102         return;
103     }
104     if(temp->left != NULL){
105         printNode(&(temp->left));
106     }
107     printf("%d|\n", temp->data->num);
108     if(temp->right != NULL){
109         printNode(&(temp->right));
110     }
111 }
112
113 void insertNode(Node** root, Node* insNode){
114     Node* temp = *root;
115     if(temp == NULL){
116         *root = insNode;
117         return;
118     }
119
120     if(temp->data->num > insNode->data->num){
121         insertNode(&(temp->left), insNode);
122     }else if(temp->data->num < insNode->data->num){
123         insertNode(&(temp->right), insNode);
124     }else{
125         return;
126     }
127 }
128
129 void deleteNode(Node** root, int num){
130     Node* temp = *root;
131     Node *p, *child, *parent, *replace, *replace_parent;
132
133     parent = NULL;
134     p = temp;
135     while( (p != NULL)&&(p->data->num != num) ){
136         parent = p;
137         if(p->data->num > num) p = p->left;
138         else p = p->right;
139     }
140     if((p->left == NULL)&&(p->right == NULL)){
141         if(parent != NULL){
142             if(parent->left == p) parent->left = NULL;
143             else parent->right = NULL;
144         }else{
145             temp = NULL;
146             *root = temp;
147         }
148     }else if((p->left == NULL) || (p->right == NULL)){
149         if(p->left != NULL) child = p->left;
150         else child = p->right;
151
152         if(parent != NULL){
153             if(parent->left == p ) parent->left=child;
154             else parent->right = child;
155         }else{
156             temp = child;
157             *root = temp;
158         }
159     }else{
160         replace_parent = p;
161         replace = p->left;
162         while(replace->right != NULL){
163             replace_parent = replace;
164             replace = replace->right;
165         }
166         if(replace_parent->left == replace)
167             replace_parent->left = replace->left;
168         else
169             replace_parent->right = replace->left;
170         p->data->num = replace->data->num;
171         p = replace;
172     }
173     free(p->data);
174     free(p);
175 }
176
177 void releaseNode(Node** root){
178     if(*root == NULL) return;
179     releaseNode(&((*root)->left));
180     releaseNode(&((*root)->right));
181
182     free((*root)->data);
183     free(*root);
184 }
185
186 Node* searchSameNode(Node** root, int num){
187     Node* temp = *root;
188     if(temp == NULL){
189         return NULL;
190     }
191
192     if(temp->data->num == num){
193         return temp;
194     }else if(temp->data->num > num){
195         searchSameNode( &(temp->left), num);
196     }else{
197         searchSameNode( &(temp->right), num);
198     }
199 }
200
201 int transferData() {
202     gpointer sendNum = &num;
203     tid = g_timeout_add(2000, cbTransferData, sendNum);
204     return 0;
205 }
206
207 gboolean cbTransferData(gpointer argv) {
208     int *pNum = (int *)argv;
209     printf("\n");
210     printf("=====\n");
211     printf("timeout %d\n", *pNum +1);
212     printf("=====\n");
213     *(int *)pNum = *(int *)pNum + 1;
214     g_source_remove(tid);
215     return (gboolean)FALSE;
216 }
```